

Python 코드리뷰어를 위한 IDE & Test 도구 사용가이드

CONTENTS

목차

CHAPTER 1

PyCharm 설치

CHAPTER 2

PyCharm 빌드 테스트

CHAPTER 3

Pytest 세팅 / 테스트

CHAPTER 4

Pytest - Code Coverage

CHAPTER 5

Pytest - Mocking

CHAPTER 6

Console 테스트 (E2E Test)

Chapter 01

PyCharm 설치

Python 코드리뷰어를 위한 Build & Test 요약

수업 환경

동탄에듀센터2 (비 보안 사업장)

Windows PC 노트북 제공

기준 PC

Knox 화상채팅 + 개발툴 + Discord에 끊김이 없는 PC 환경

- Intel 8세대 i5 이상 사양
- RAM 16GB 이상
- Windows 10 이상

코드리뷰어 교육은 동일한 환경에서 수업을 진행합니다.

제공되는 Windows 노트북으로 수업이 진행됩니다.

개인 노트북 사용으로 인한 환경 세팅 에러 해결에 도움을 드릴 수 없습니다.

팀원들간 IDE의 프로젝트 공유하게 됨

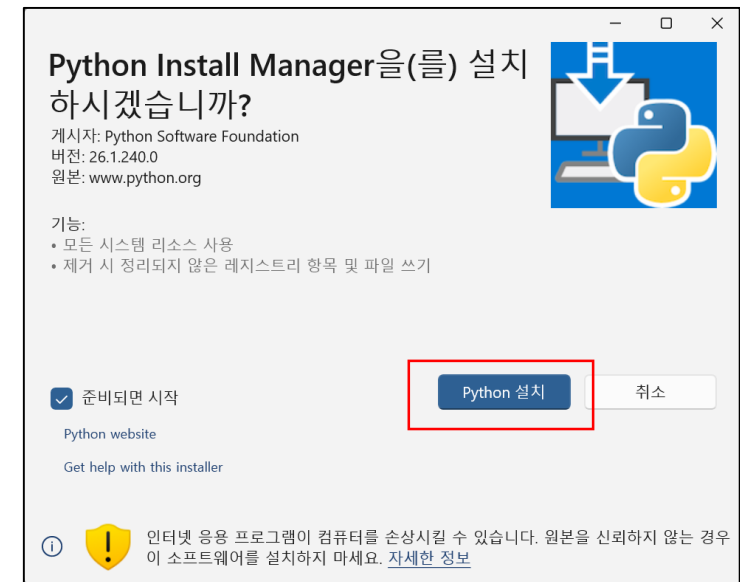
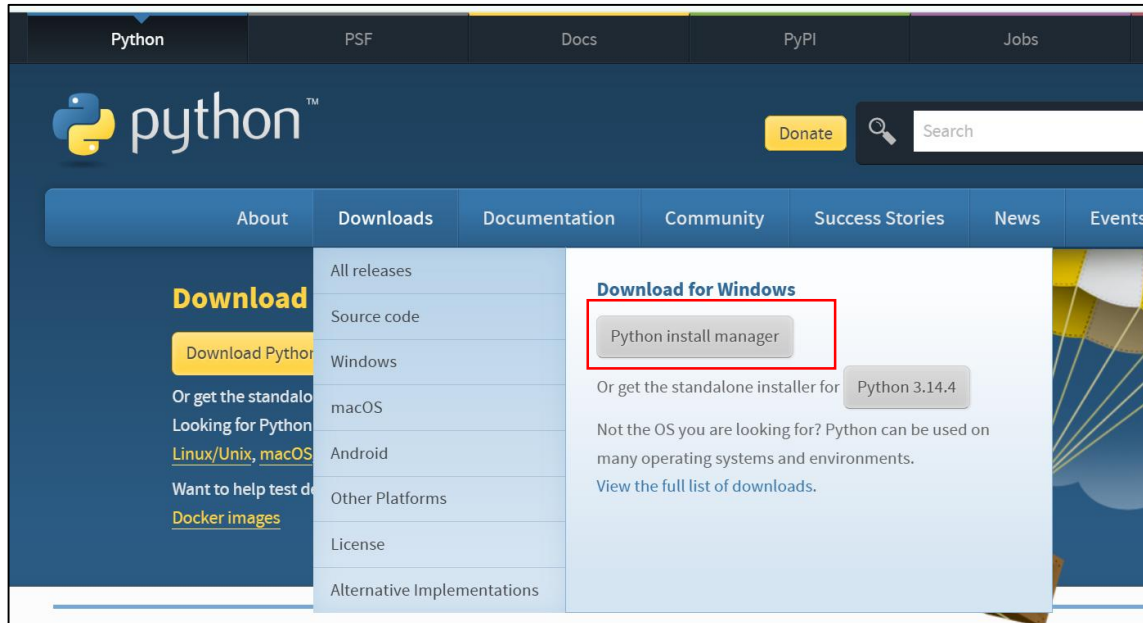
PyCharm (2025년 이후 버전)

- Python 진영에서 가장 많이 사용되는 IDE는 vscode 이지만, 리팩토링 도구, 테스트 도구 등을 지원하기에 PyCharm 을 사용한다.
- 다운로드 : <https://www.jetbrains.com/pycharm/>
 - 25년 이후에는 Community 버전 / Professional 버전 구분은 없어졌다.

Python 버전 - Python 3.14

Python 설치

Python 3.14 버전 다운로드 / 설치



Python 설치

PATH 등록 --> y 입력

CPython 설치 ---> y 입력

```
C:\Program Files\WindowsA\ >
Welcome to the Python installation manager configuration helper.

*****

The global shortcuts directory is not configured.

Configuring this enables commands like python3.14.exe to run from your
terminal, but is not needed for the python or py commands (for example, py
-V:3.14).

We can add the directory (C:\Users\minco\AppData\Local\Python\bin) to PATH now,
but you will need to restart your terminal to use it. The entry will be removed
if you run py uninstall --purge, or else you can remove it manually when
uninstalling Python.
Add commands directory to your PATH now? [y/N] y
```

```
C:\Program Files\WindowsA\ >
The global shortcuts directory is not configured.

Configuring this enables commands like python3.14.exe to run from your
terminal, but is not needed for the python or py commands (for example, py
-V:3.14).

We can add the directory (C:\Users\minco\AppData\Local\Python\bin) to PATH now,
but you will need to restart your terminal to use it. The entry will be removed
if you run py uninstall --purge, or else you can remove it manually when
uninstalling Python.
Add commands directory to your PATH now? [y/N] y
PATH has been updated, and will take effect after opening a new terminal.

*****

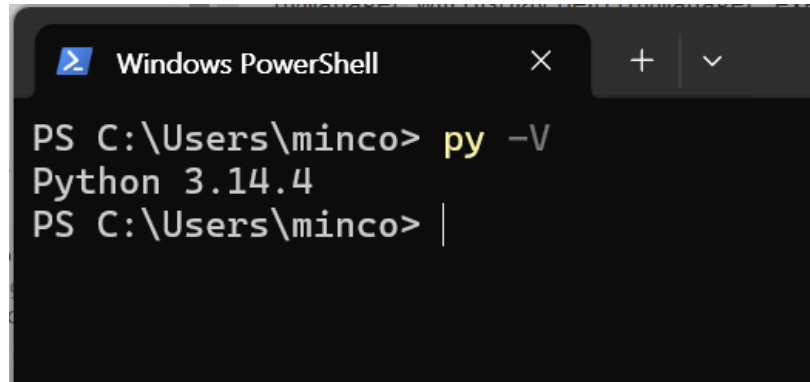
You do not have the latest Python runtime.

Install the current latest version of CPython? If not, you can use 'py install
default' later to install.

Install CPython now? [Y/n] y
Python install manager was successfully updated to 26.1.
```

설치 확인

PowerShell 실행 후 `py -V` 로 설치 확인



```
Windows PowerShell
PS C:\Users\minco> py -V
Python 3.14.4
PS C:\Users\minco> |
```

Pycharm 다운로드

26년 이후, Ultimate / Community 버전이 통합되었습니다.

30일간 체험판이 끝나면, 자동으로 Community 버전으로 전환되어 동작합니다.

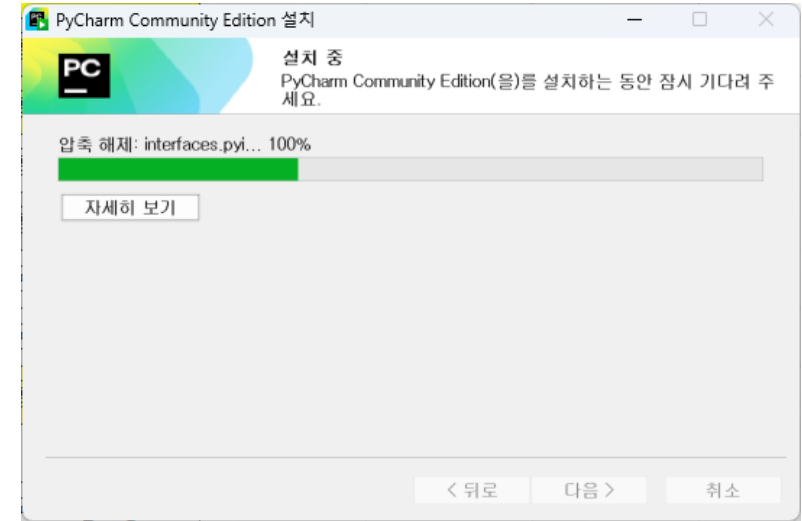
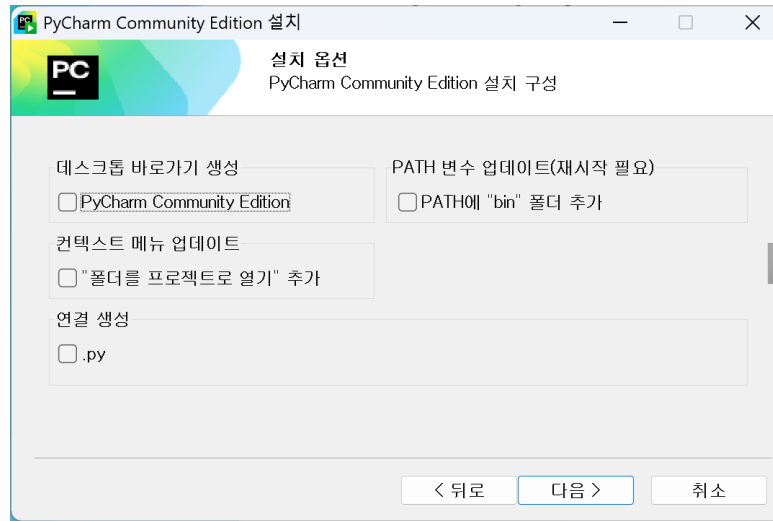
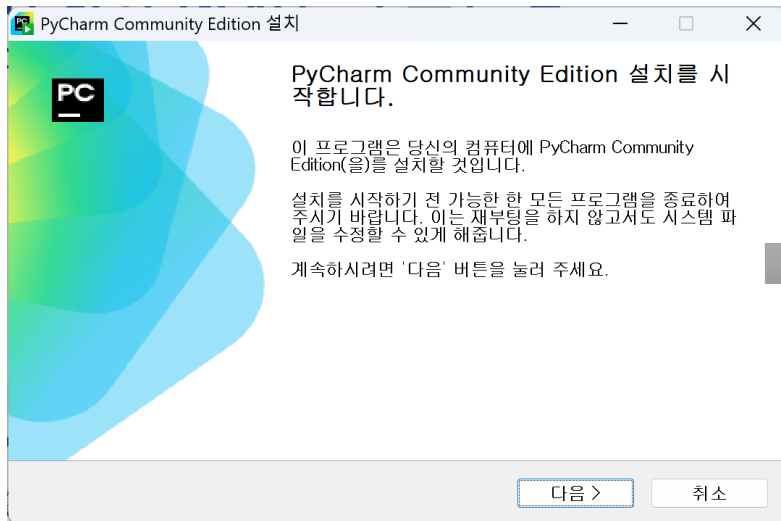


설치하기

다른 옵션지정 없이 Next만 누르면 끝

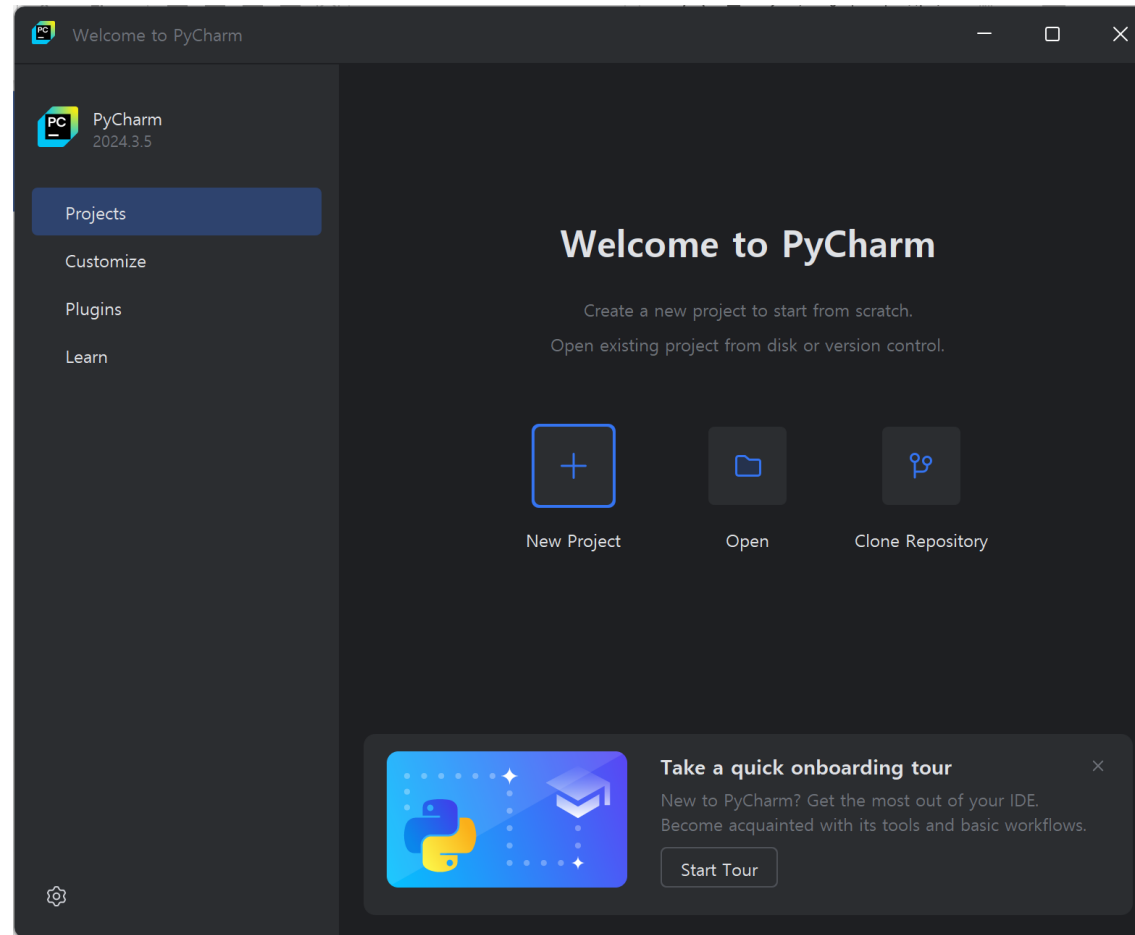
다운로드 소요시간 : 약 1분 (1GB 이내)

설치 소요시간 : 약 3분



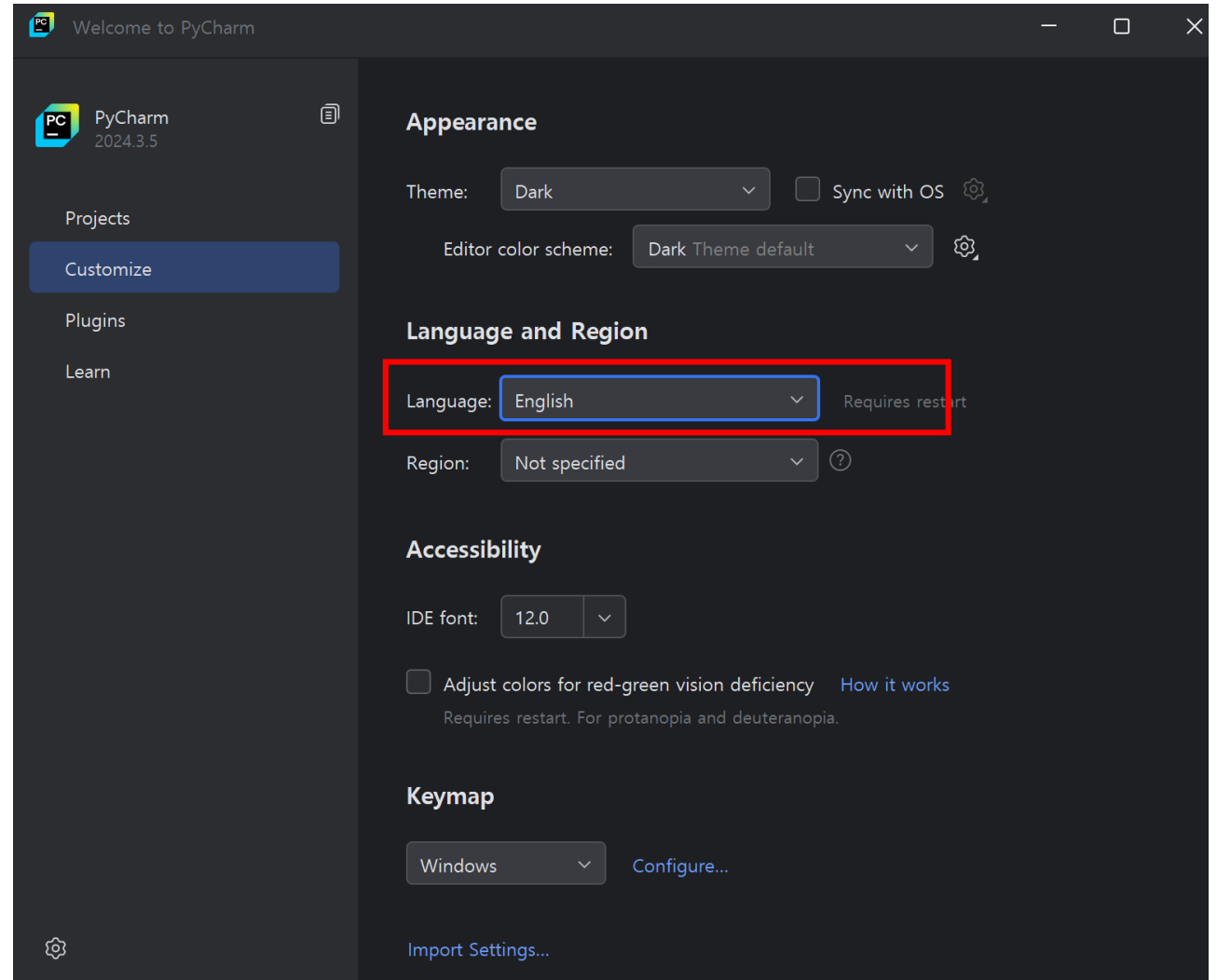
설치 완료후 실행화면

설치 완료 후, 실행화면



IDE - English 로 세팅

수업에 사용되는 IDE는
영어로 설정하여 진행됩니다.

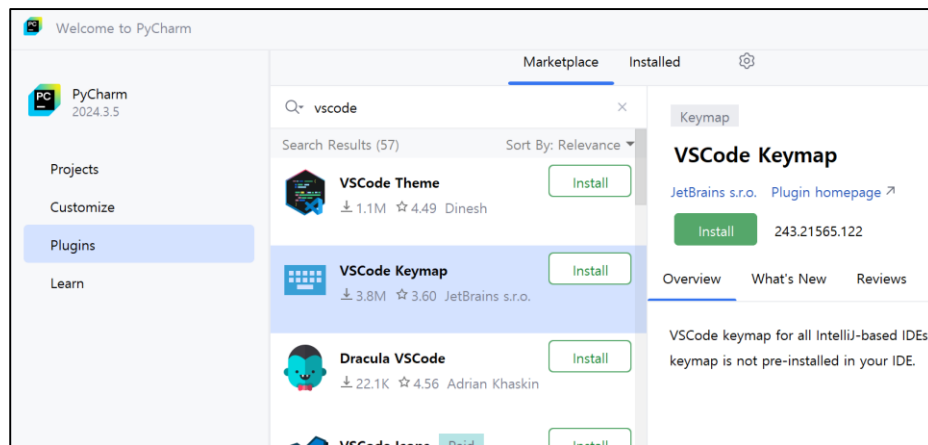


[참고] 개인이 희망하는 설정으로 세팅

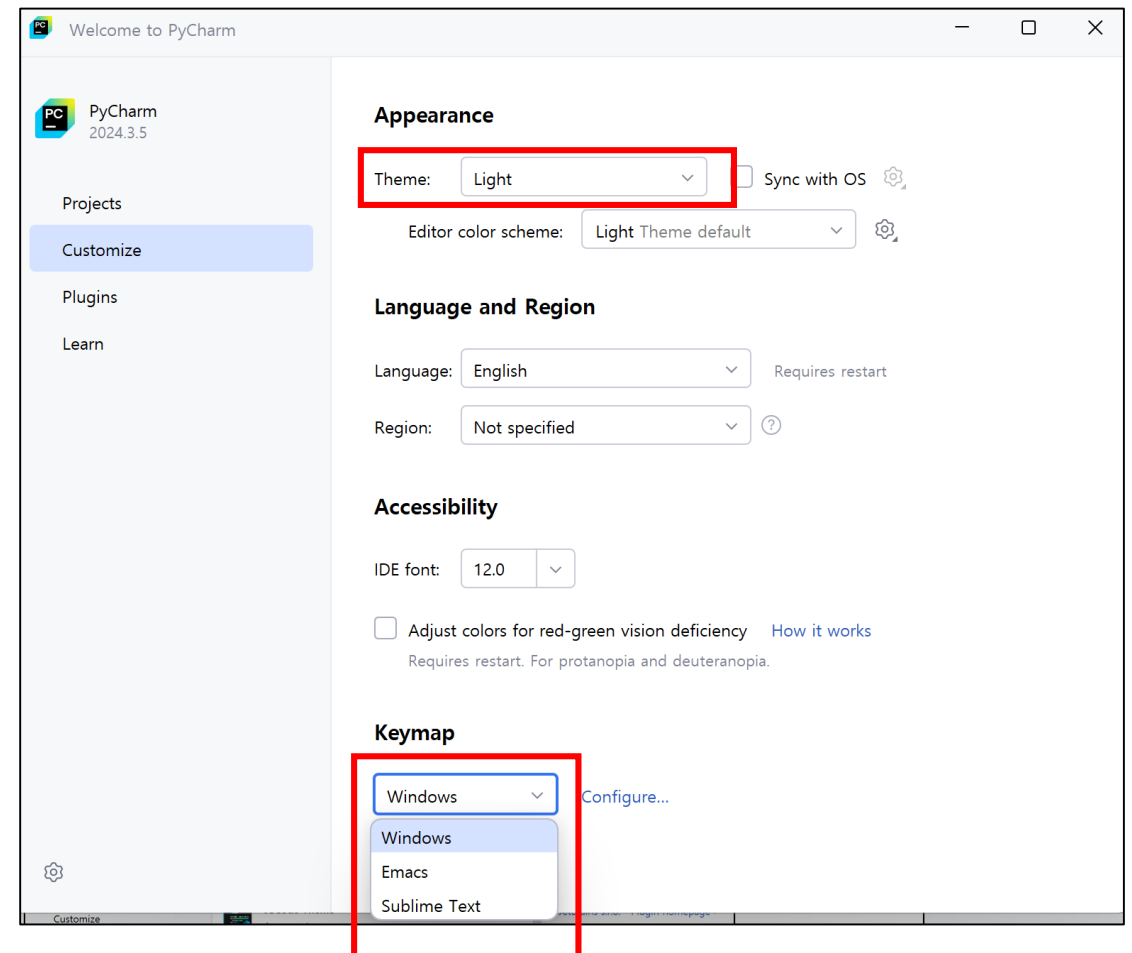
테마 : 어두운 / 밝음 설정

(단축키)키맵 : 본인의 편한 단축키 셋

VSCode가 편한 사용자
: 플러그인 > 설치하여 설정



(옵션) Visual Studio Code가 편리하신 분은 따로 플러그인 설치



Chapter 02

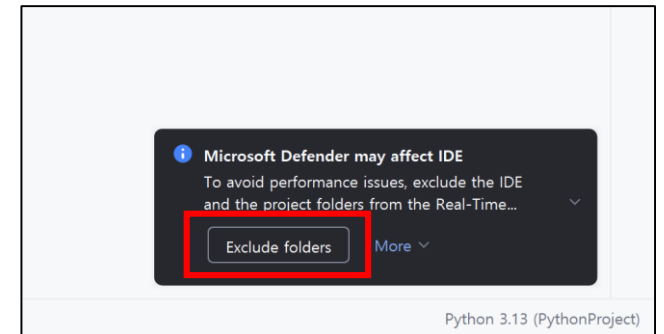
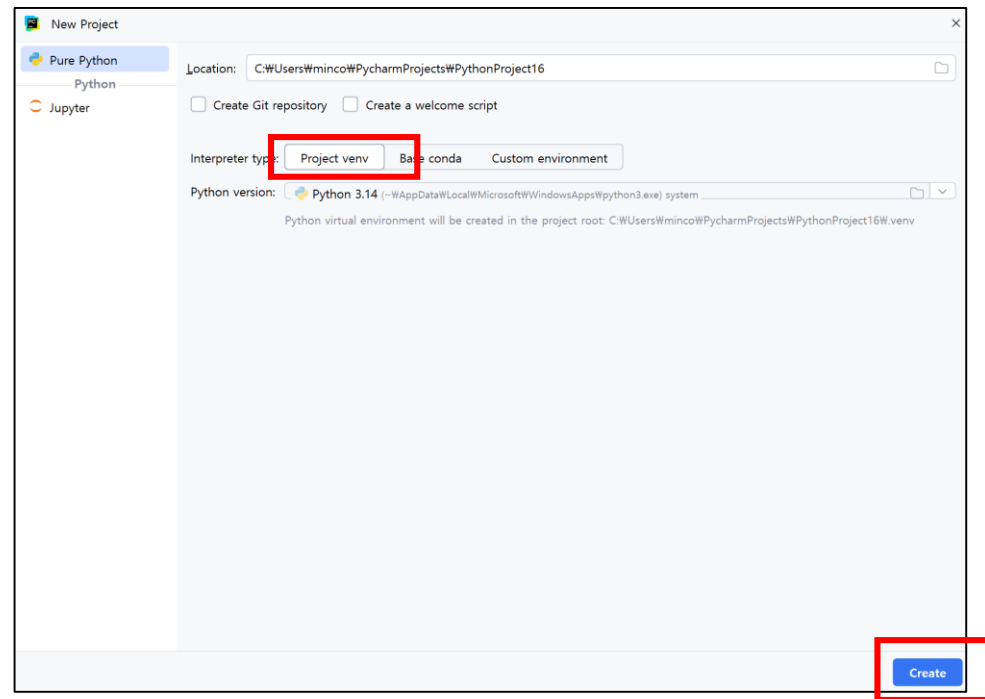
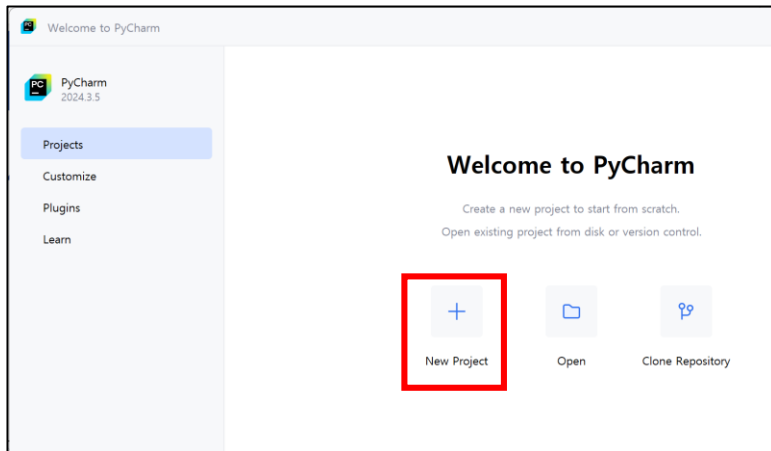
PyCharm 빌드 테스트

Python 코드리뷰어를 위한 Build & Test 요약

프로젝트 생성

기본 프로젝트 생성

venv 샌드박스 안에서 파이썬이 실행되게끔 venv를 선택 후 Create를 누른다.

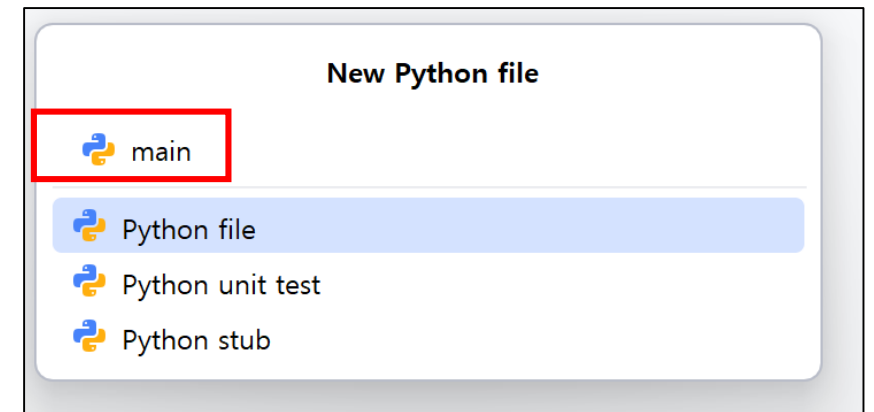
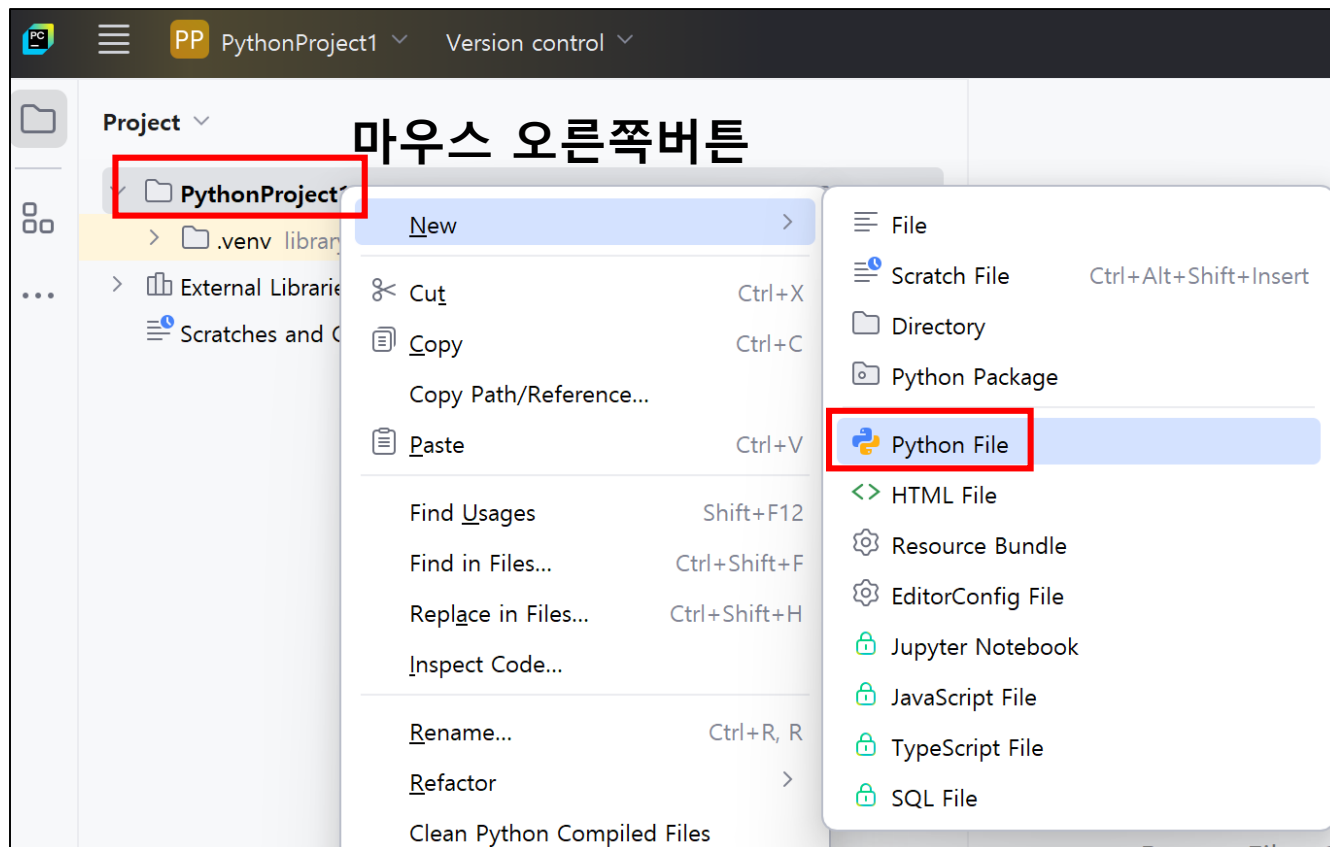


이 창이 뜬다면 Exclude를 누른다.
Windows 보안해제를 해야
빌드속도가 더 빠르다.

빌드 테스트 1

py 파일을 생성한다.

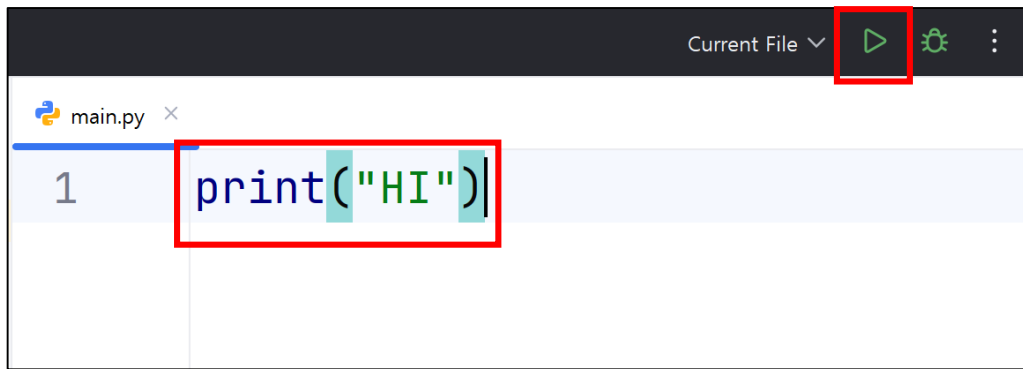
반드시 .venv 밖에 만든다.



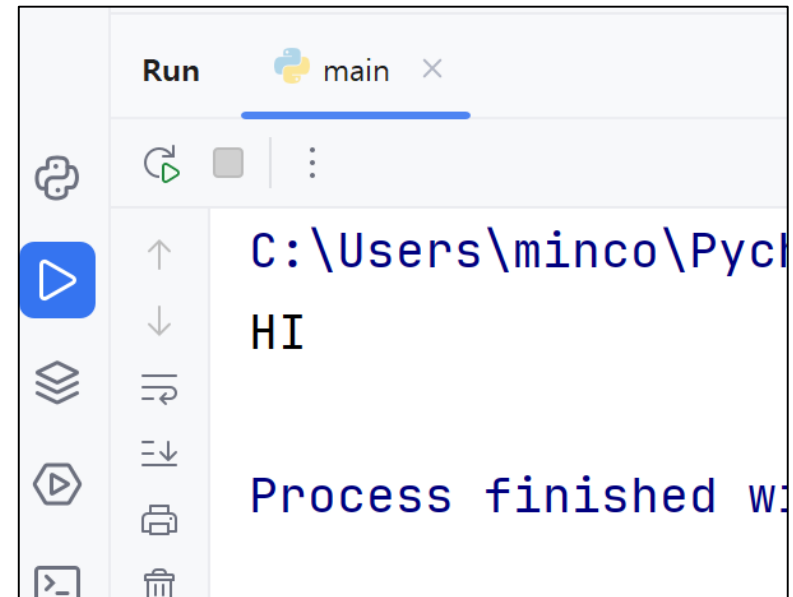
main.py 파일 생성

빌드 테스트 2 (완료)

기본 코드 작성 후 Run 해보기

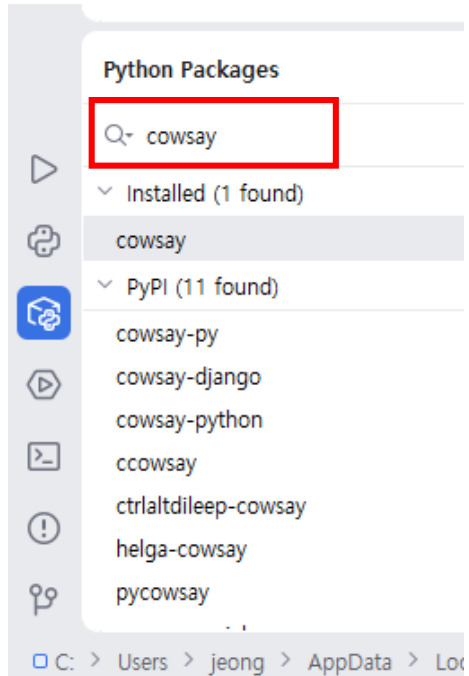


```
1 print("HI")
```

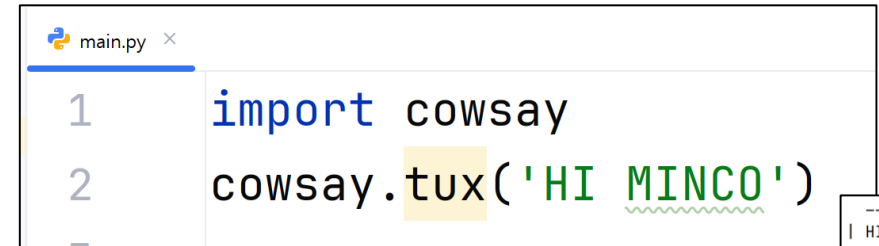


Library 설치 테스트

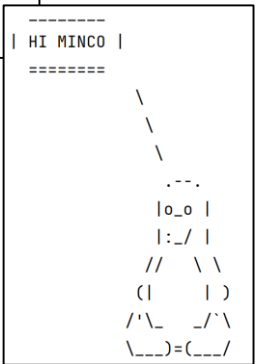
보안프로그램으로 인해
패키지 설치에 이상 없는지 확인한다.



Python packages 를 이용해서 라이브러리 설치



에러가 보이지 않으면 된다.



Chapter 03

Pytest 세팅 / 테스트

C++ 코드리뷰어를 위한 Build & Test 요약

Python 진영에서 가장 많이 사용되는 Unit Test Framework

- 파이썬에는 "unittest" 라는 Unit Test Framework가 기본적으로 제공되지만 파이썬 진영에서는 pytest 라이브러리를 더 많이 사용함
- 별도 설치 필요
 - `pip install pytest`
- pytest 3.8 이상 버전에서 동작

pytest 사용 가이드 (Github & Document)

- <https://docs.pytest.org/en/stable/>
- <https://github.com/pytest-dev/pytest>

unittest VS pytest

unittest

- 파이썬 표준 Library에 포함되어 있음
 - 별도 설치 필요 없음
- xUnit 스타일의 코드 (gTest, JUnit 등)
 - xUnit 스타일의 UnitTest 경험자들은, 손쉽게 손쉽게 사용 가능

pytest

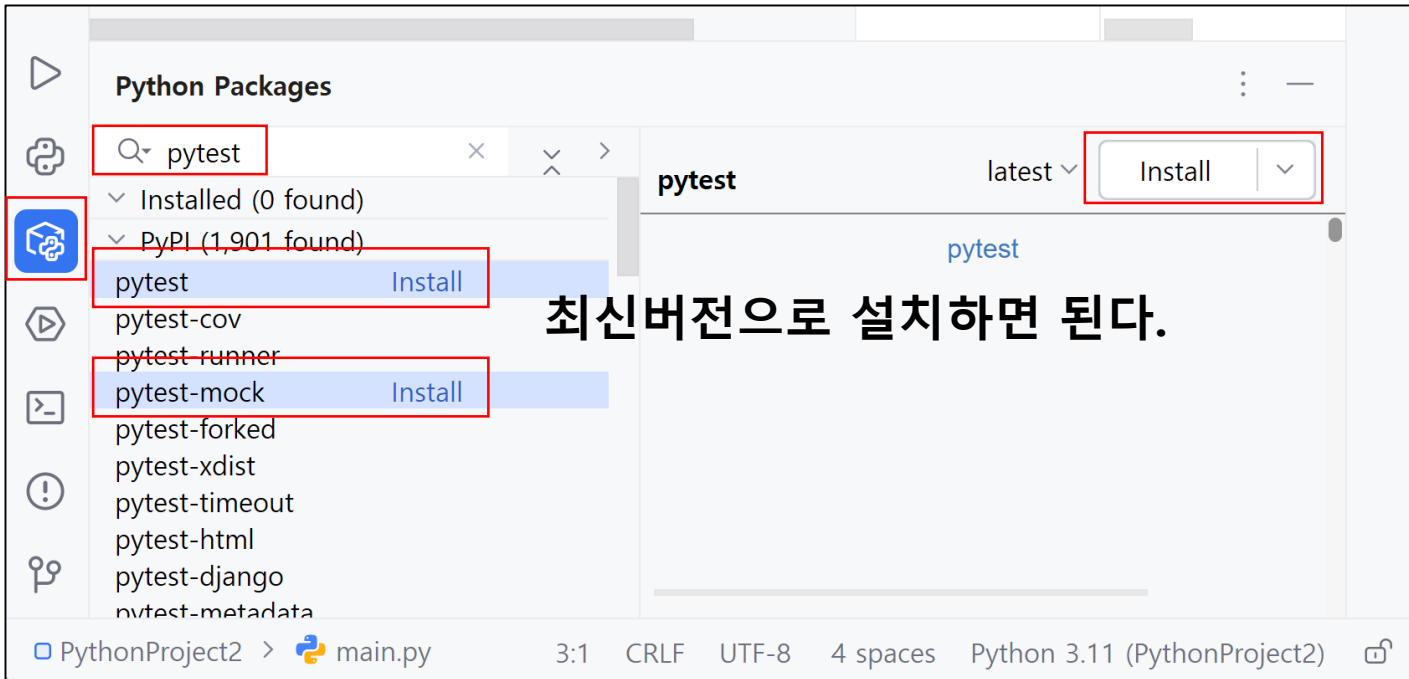


- 2010년 10월 공식 버전 출시
- 더 파이썬스러운 테스트 코드 작성 가능
 - 더 파이썬스러운 코드로 테스트 진행
- 별도 설치 필요
 - UnitTest : pip install pytest
 - Mocking : pip install pytest-mock

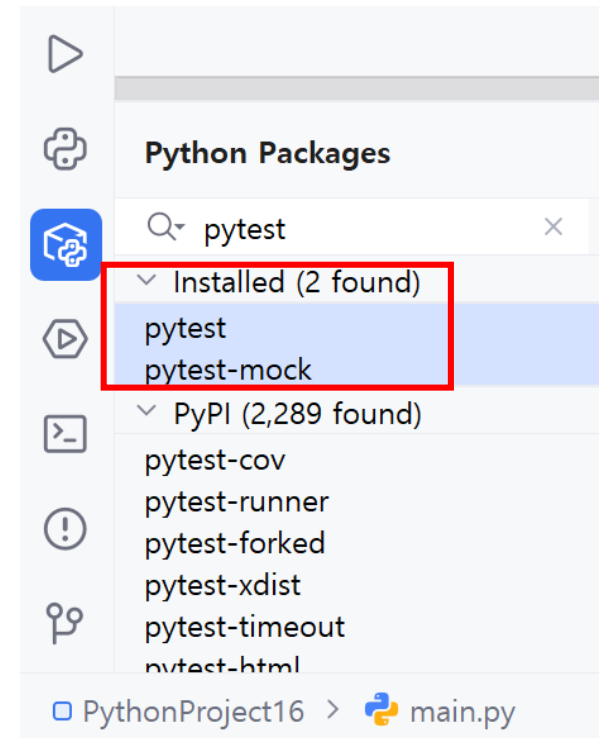
pytest 설치

pytest 패키지와 pytest-mock 플러그인을 설치한다

- pytest : 유닛테스트 **패키지**
- pytest-mock : pytest에 Mocking 기능을 추가해주는 **플러그인**



pytest와 pytest-mock을
각각 install 버튼을 눌러 설치



설치가 되었음을
이곳에서 확인할 수 있다.

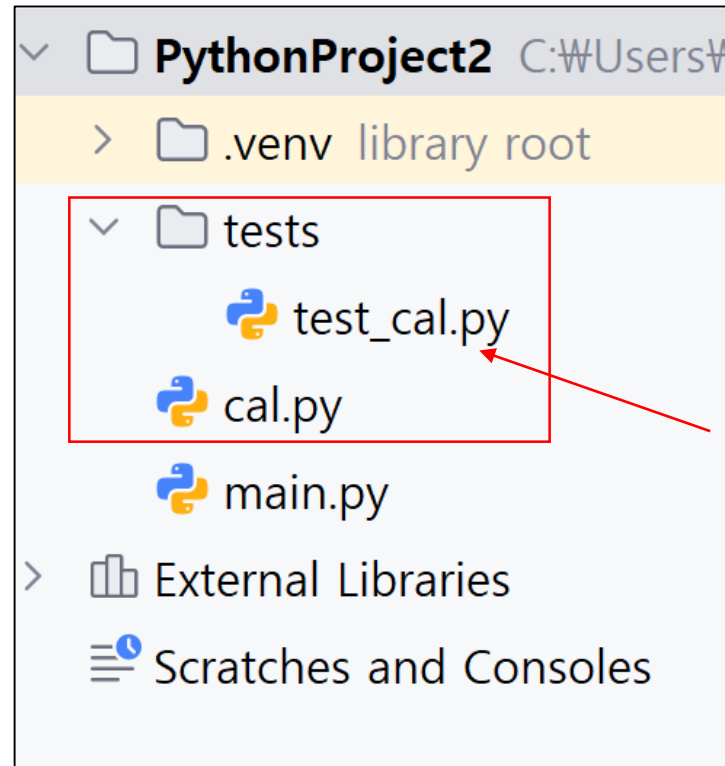
pytest 세팅하기

Directory를 추가한다.

- tests 폴더 : 테스트 파일을 넣는 곳

2개의 py 파일을 생성한다.

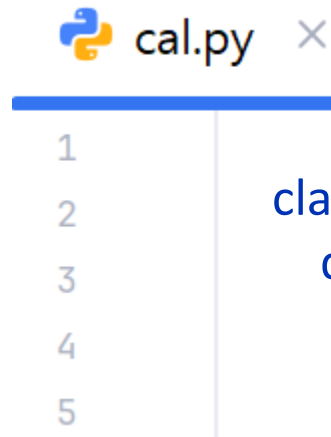
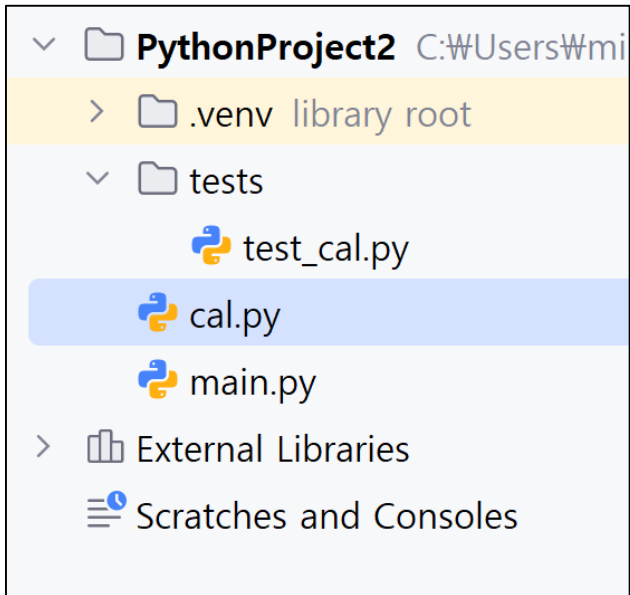
- cal.py 파일을 생성
- test_cal.py 파일을 tests 폴더에 생성



테스트 파일명은
반드시
"test_" 로 시작해야
테스트 파일로
인식함

테스트 대상이 될 소스코드 입력하기

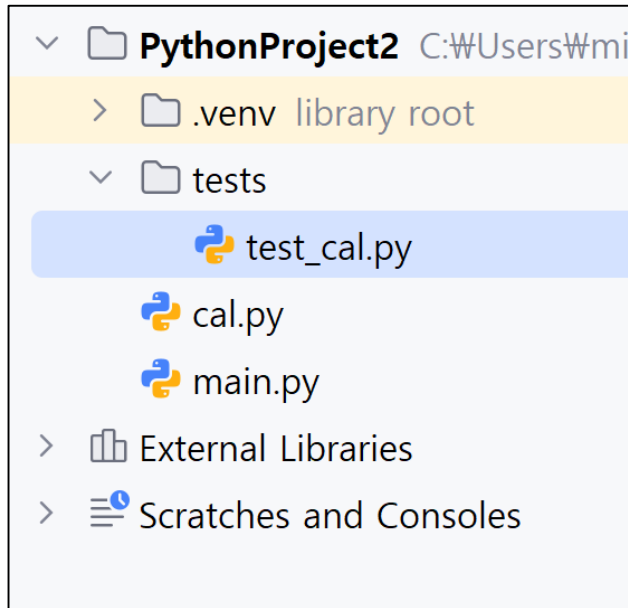
cal.py 파일에 기본 코드를 입력한다.



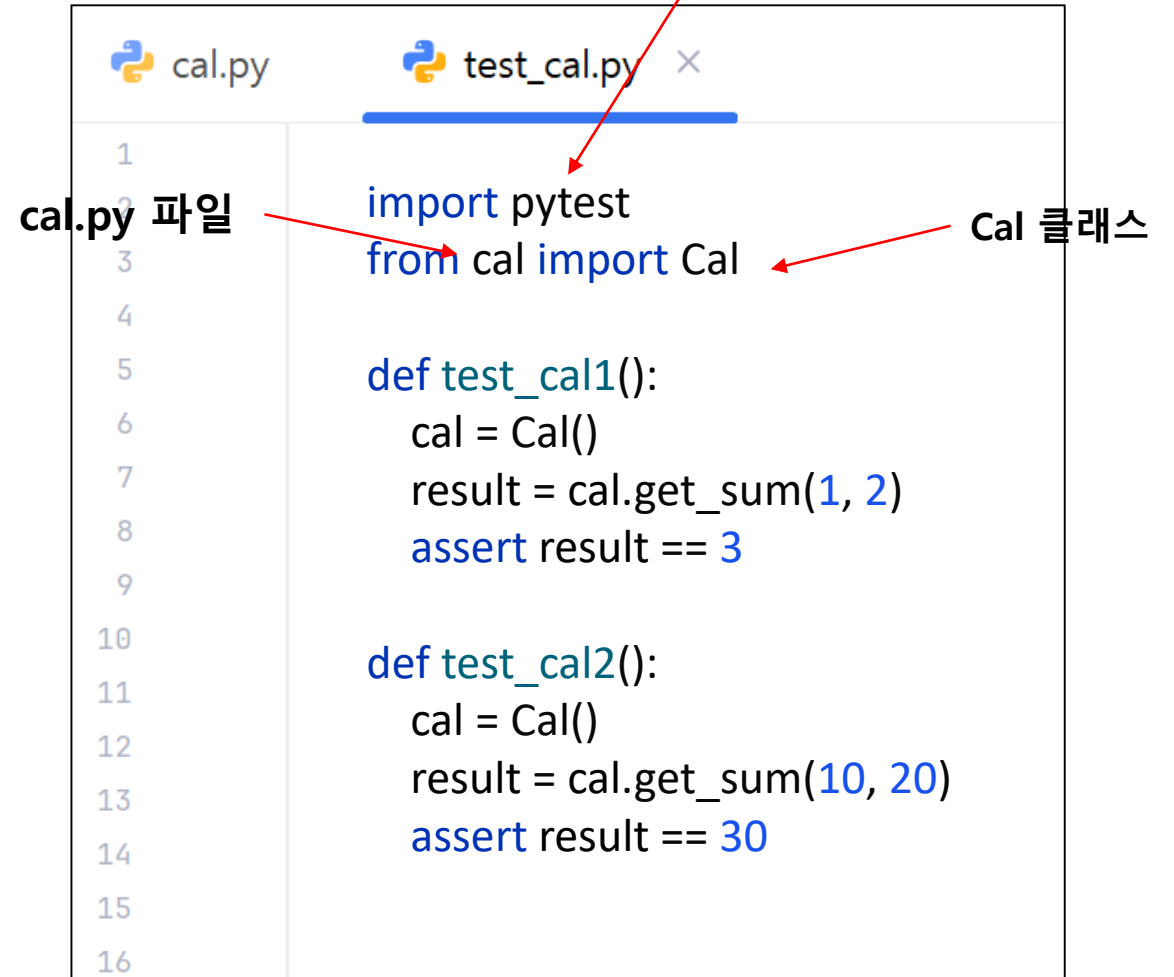
```
class Cal:  
    def get_sum(self, a: int, b: int) -> int:  
        return a + b;
```

테스트 코드 삽입

테스트 코드 삽입하기

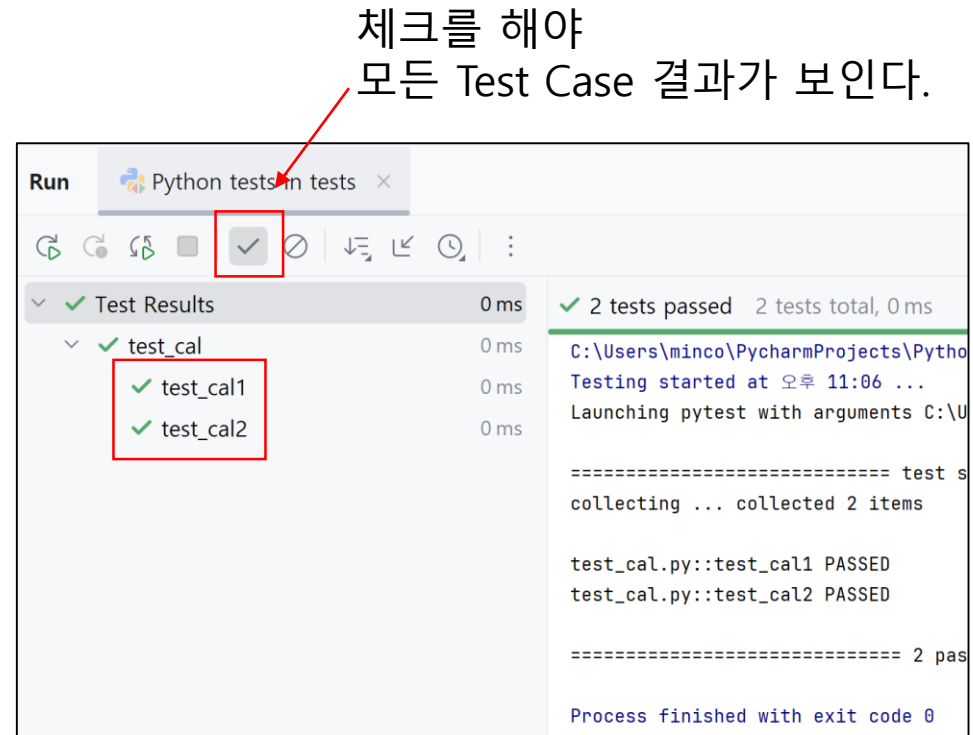
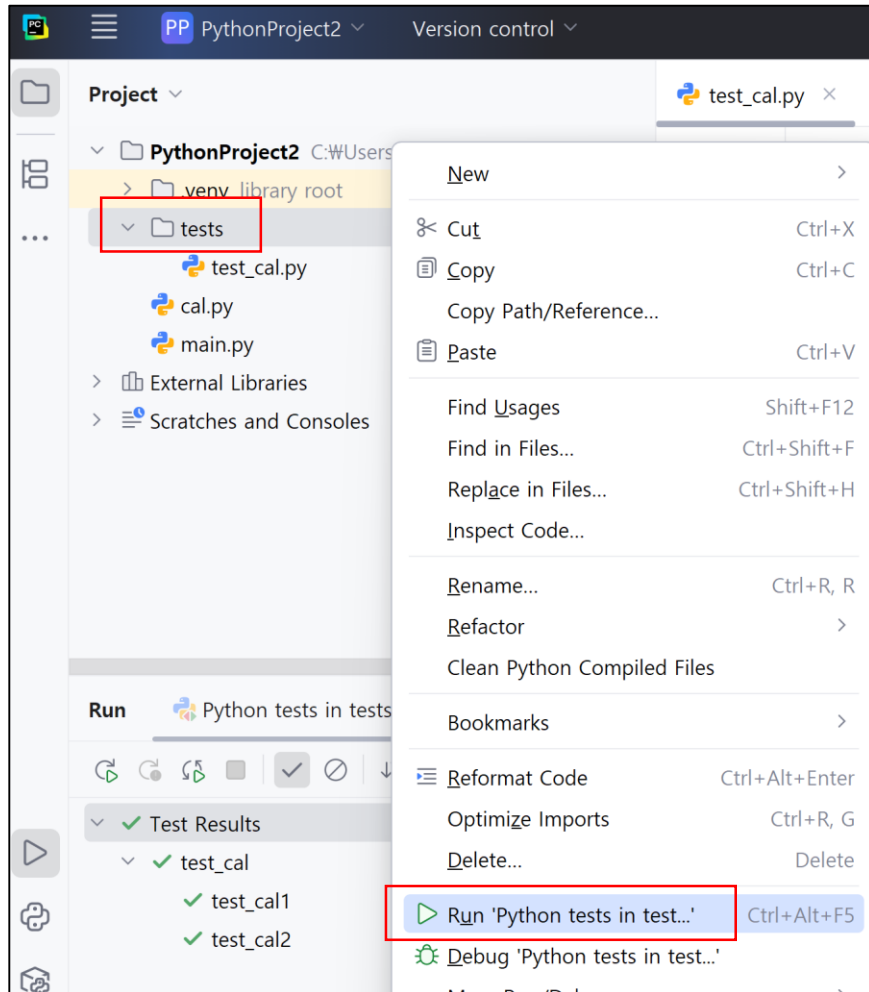


[권장사항]
pytest 임을 명시하기 위해
항상 import pytest를 써주자



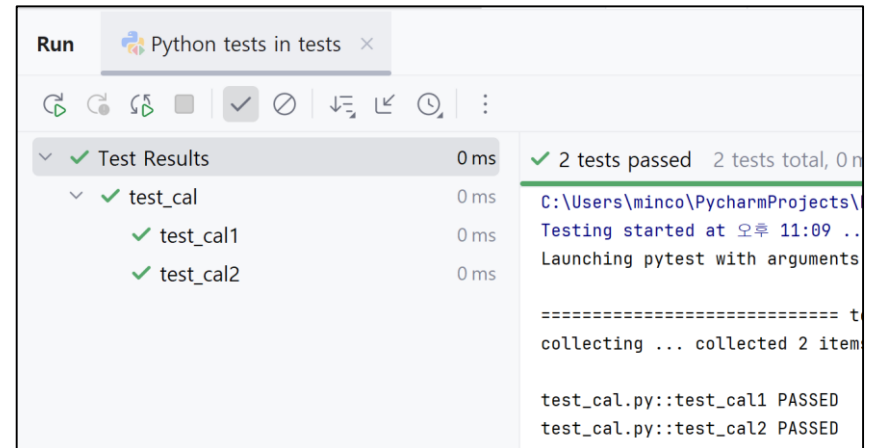
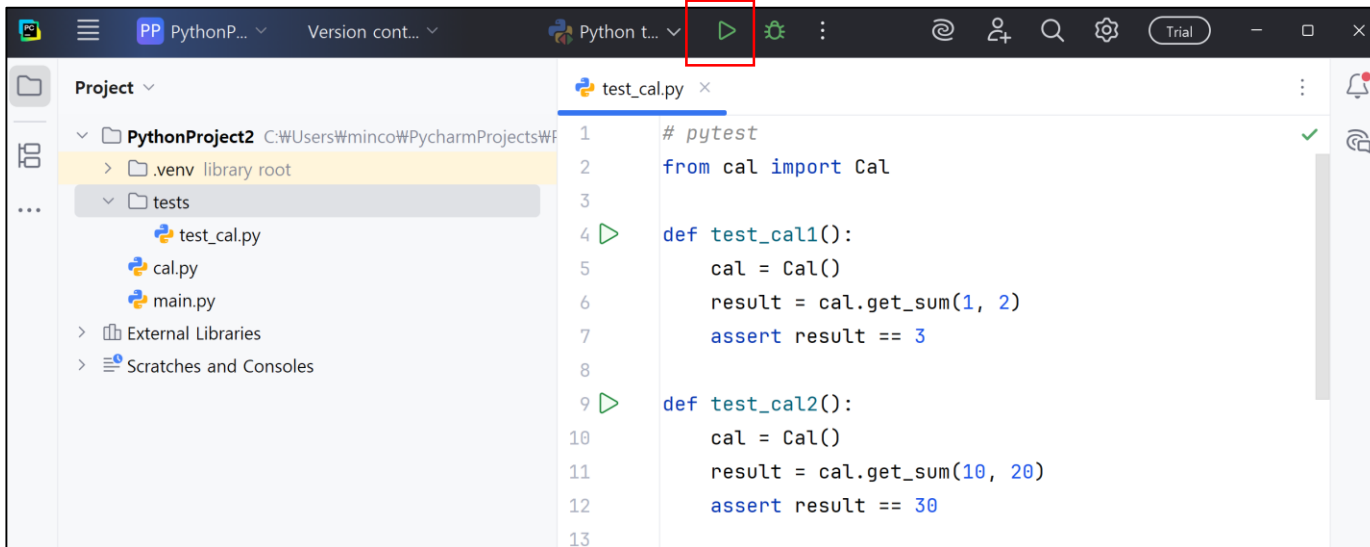
테스트 RUN - 1회성 세팅

tests 폴더로 테스트를 하면, 모든 테스트를 수행할 수 있다.



테스트 RUN

이후에는 RUN 버튼만 누르면
모든 테스트가 수행된다.



Chapter 04

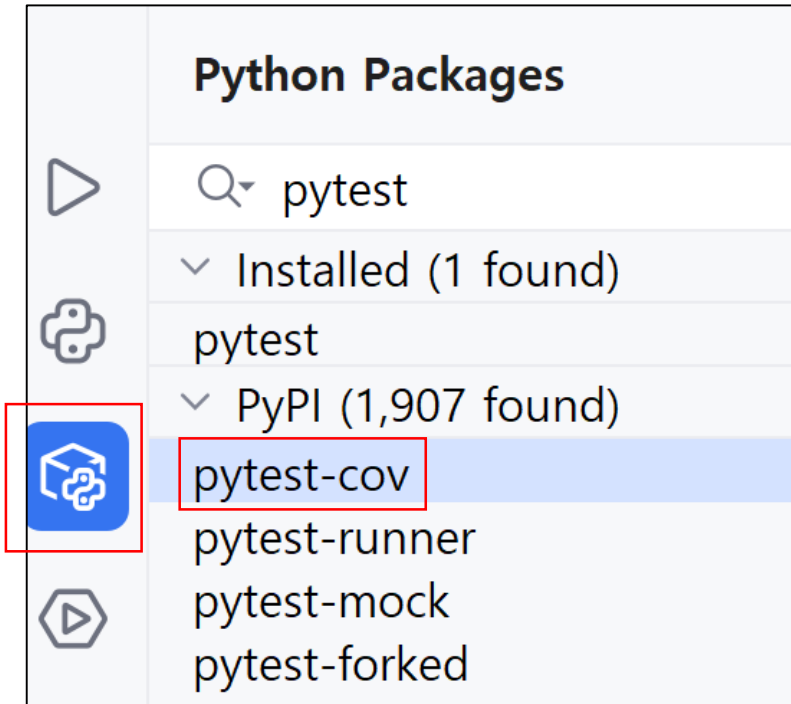
Pytest - Code Coverage

Python 코드리뷰어를 위한 Build & Test 요약

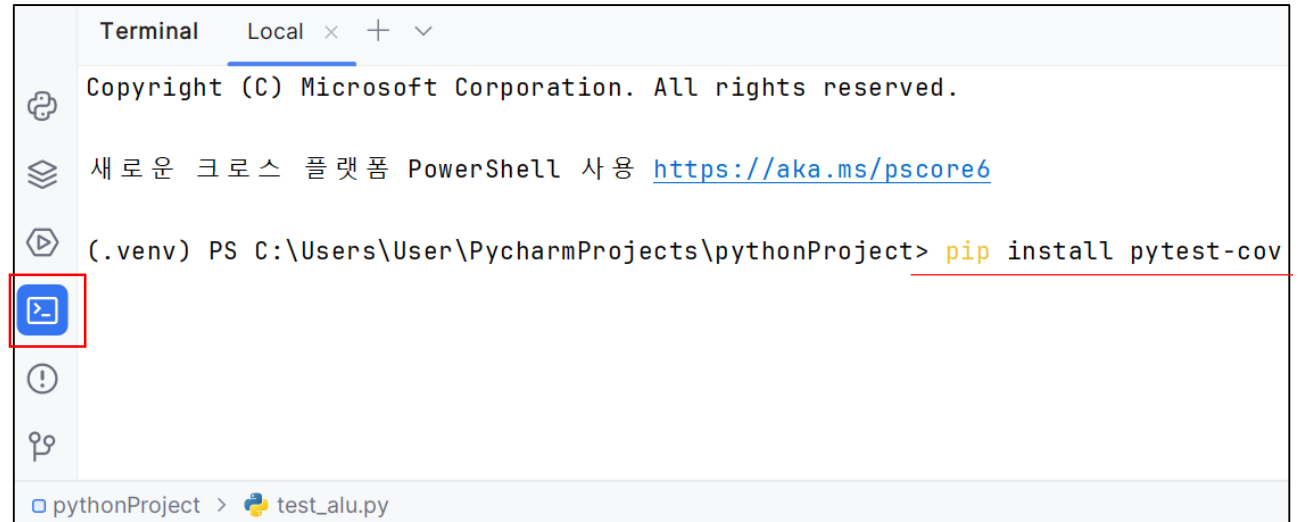
코드 커버리지 측정도구 설치

pytest의 커버리지 측정 플러그인

- 설치방법 1 : 아래 그림처럼, GUI로 설치
- 설치방법 2 : `pip install pytest-cov`



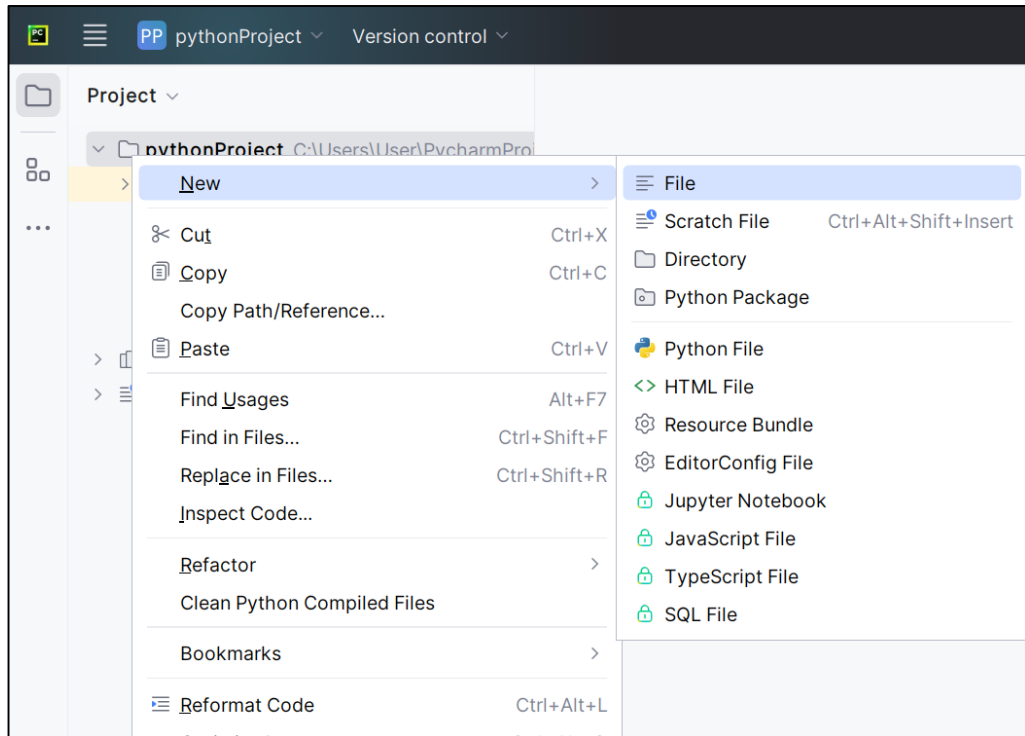
설치방법 1
선택 후 install 버튼 클릭



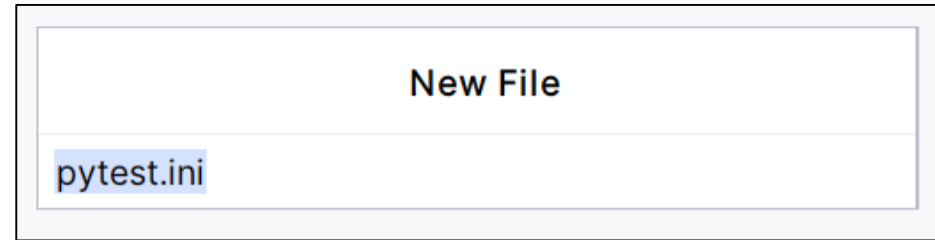
설치방법 2
`pip install pytest-cov`

[필수] 커버리지 측정을 위해 pytest.ini 파일 추가

pytest를 터미널에서 실행할 수 있도록 ini 파일을 추가한다.
폴더 구조를 인식하고, 결과 파일을 html로 생성



프로젝트 마우스 우측버튼 > New > File



pytest.ini 파일 추가



```
[pytest]
pythonpath = .
addopts = -s --cov-report=html
```

pytest의 프로젝트 구조 인식 &
coverage 측정을 위한 설정 추가

<https://gist.github.com/mincoding1/d4d6ed9844cce0b7d940da422c35fd94>

pytest.ini

소스코드 경로에 따라, Path 경로를 수정해준다.

```
pytest.ini x
1 [pytest]
2 pythonpath = .
3 addopts = -s --cov-report=html
4
```

프로젝트 루트 폴더에 소스코드가 있는 경우

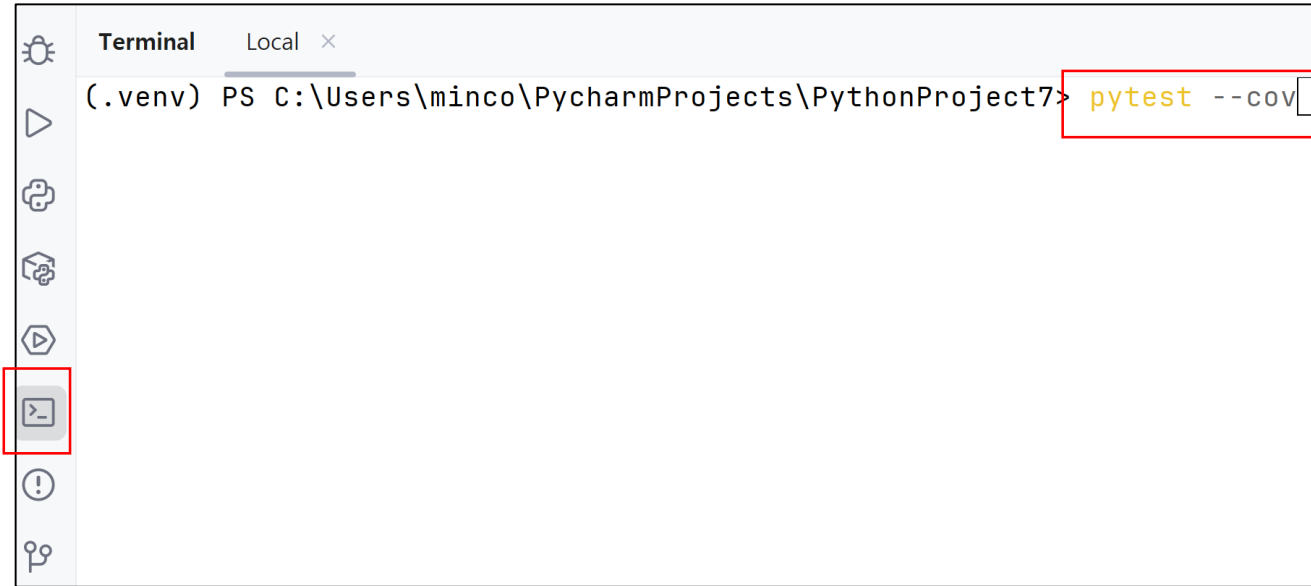
```
pytest.ini x
1 [pytest]
2 pythonpath = src
3 addopts = -s --cov-report=html
4
```

src 폴더에 소스코드가 있는 경우

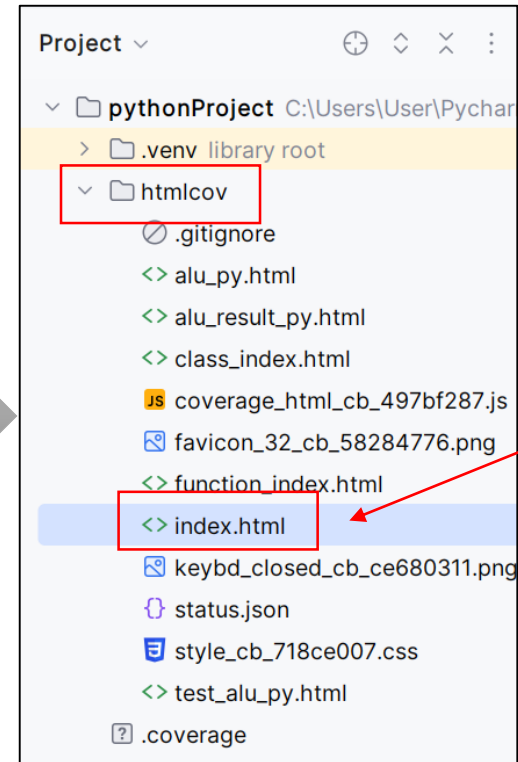
코드 커버리지 측정 도구 실행

터미널에서 직접 실행

pytest --cov 라고 입력하면, Coverage 측정 결과가 **index.html**에 기록됨



pytest --cov 명령어 입력

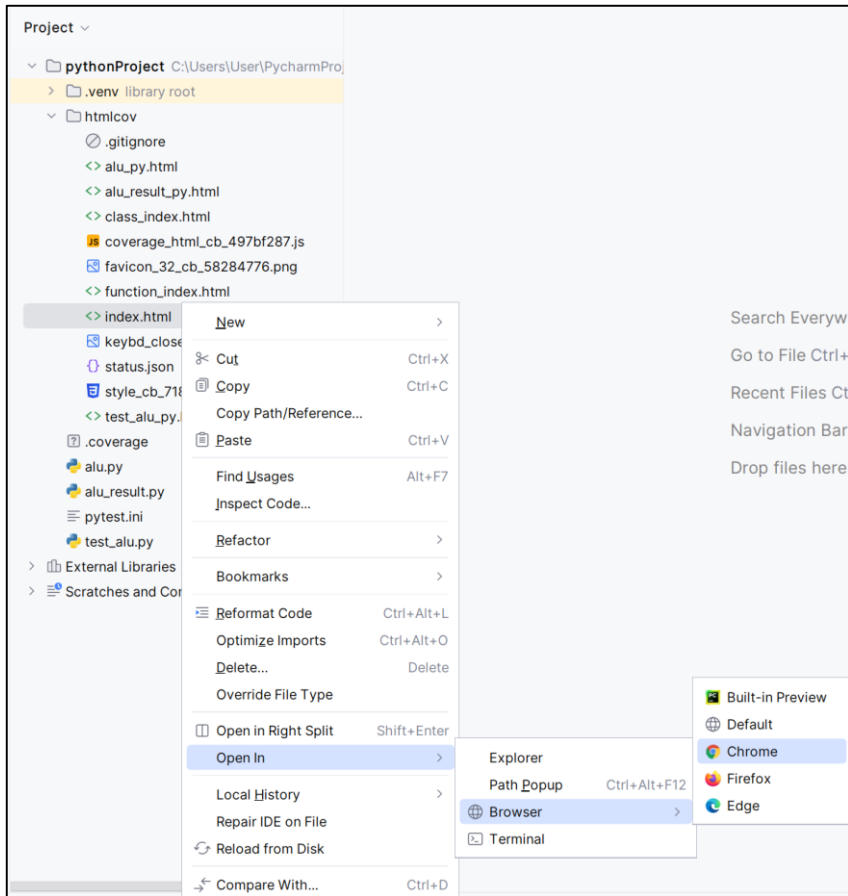


index.html
Coverage 측정
결과파일이 생성된다.

코드 커버리지 측정 결과 확인

line coverage 측정 결과 확인

- Test 코드가 닿지 못한 부분은 빨간색으로 표기된다.



브라우저에서 열기

Line Coverage

Coverage report: 100%

Files Functions Classes

coverage.py v7.13.5, created at 2026-05-03 02:10 +0900

File	statements	missing	excluded	coverage
cal.py	3	0	0	100%
tests\test_cal.py	10	0	0	100%
Total	13	0	0	100%

coverage.py v7.13.5, created at 2026-05-03 02:10 +0900

측정 결과 확인

Coverage for cal.py: 100%

3 statements 3 run 0 missing 0 excluded

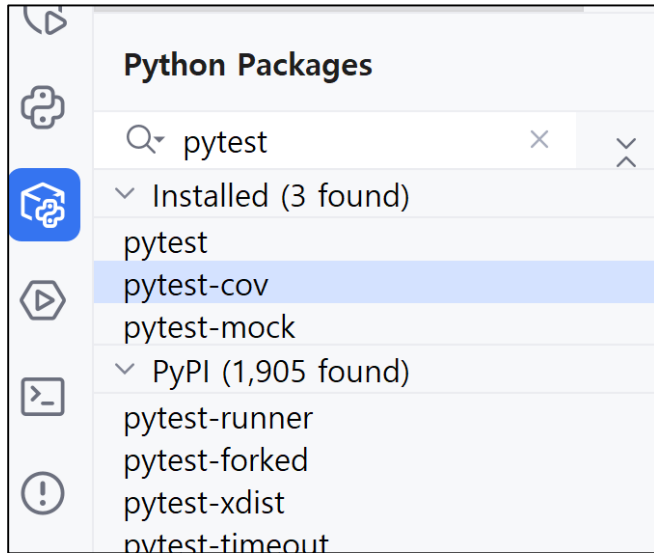
« prev ^ index » next coverage.py v7.13.5, created at 2026-05-03 02:09 +0900

```
1 class Cal:
2     def get_sum(self, a: int, b: int) -> int:
3         return a + b;
```

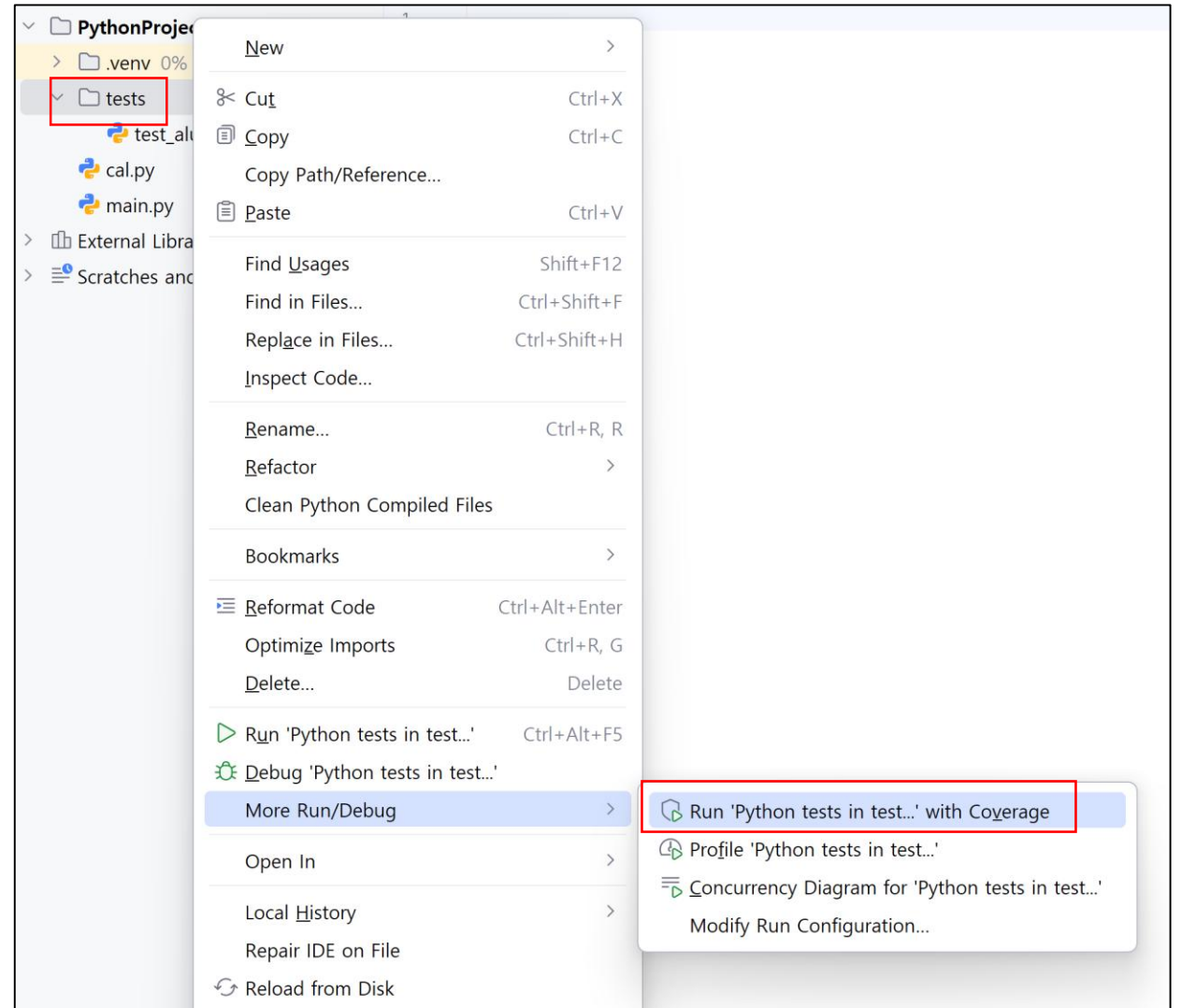
« prev ^ index » next coverage.py v7.13.5, created at 2026-05-03 02:09 +0900

[참고] 유료버전 PyCharmCode Coverage 측정방법

만약 PyCharm Enterprise 유료버전인 경우
pytest-cov 설치 후
바로 Coverage 측정 가능



pytest로 검색
> pytest-cov 플러그인 설치



Chapter 05

Pytest - Mocking

C++ 코드리뷰어를 위한 Build & Test 요약

Mocking 기본 코드 (기본 Stub)

pytest-mock 플러그인을 설치하면
mockers를 통해 mocking이 가능하다.

```
test_cal.py x
1 import pytest
2 from cal import Cal
3
4 def test_cal_mock(mock):
5     cal = Cal()
6
7     mock = mock.patch.object(Cal, 'get_sum', return_value=999)
8
9     print()
10    print(cal.get_sum(1, 2))
11    print(cal.get_sum(1, 4))
12
13    assert mock.call_count == 2
14
```

함수를 Mocking할 때는 생략하지만,
Custom 객체를 Mocking하는 경우는
Object라고 기입해야 함

patch : 부분변경이라는 의미
(ex. 패치파일)
mock 객체가 살아있는 Scope에서는
모두 patch한대로 부분 변경이 일어난다.



```
Run Python tests in tests x
[Icons]
Test Results 0 ms ✓ 1 test passed 1 te
  test_cal 0 ms
    test_cal_mock 0 ms
C:\Users\minco\Pyth
Testing started at
Launching pytest wi
=====
collecting ... coll
test_cal.py::test_c
999
999
```

Mocking 기본 코드 (특정 값에 대해 Stub 걸기)

특정 값으로 함수 호출했을 때만,
정해진 값으로 반환하는 Stub 걸기

patch 하기 전 상태의 함수를
백업한다.

Cal.get_sum을 사용하면
내부 원리상 무한재귀가 발생하므로,
백업 함수를 사용

```
import pytest
from cal import Cal

def test_cal_mock(mock):
    cal = Cal()

    origin_get_sum = Cal.get_sum

    def response_handler(self, a, b):
        if (a, b) == (1, 2):
            return 999
        return origin_get_sum(self, a, b)

    mock = mock.patch.object(
        Cal,
        'get_sum',
        side_effect=response_handler,
        autospec=True
    )

    print()
    print(cal.get_sum(1, 2)) # 999
    print(cal.get_sum(1, 4)) # 5

    assert mock.call_count == 2
```

행동 검증하기

특정 함수 호출이
몇 회 호출되었는지 검증 방법

소스코드

<https://gist.github.com/mincoding-ai/f1300d618f2e263d25aba28e25ca8021>

정확히 (1, 2)가
몇번 호출되었는지 검증

```
import pytest
from cal import Cal

def test_cal_mock(mock):
    cal = Cal()

    origin_get_sum = Cal.get_sum

    def response_handler(self, a, b):
        if (a, b) == (1, 2):
            return 999
        return origin_get_sum(self, a, b)

    mock = mock.patch.object(
        Cal,
        'get_sum',
        side_effect=response_handler,
        autospec=True
    )

    print()
    print(cal.get_sum(1, 2))
    print(cal.get_sum(1, 1))
    print(cal.get_sum(3, 4))
    print(cal.get_sum(1, 2))

    calls = [call.args[1:] for call in mock.call_args_list]
    assert calls.count((1, 2)) == 2
```

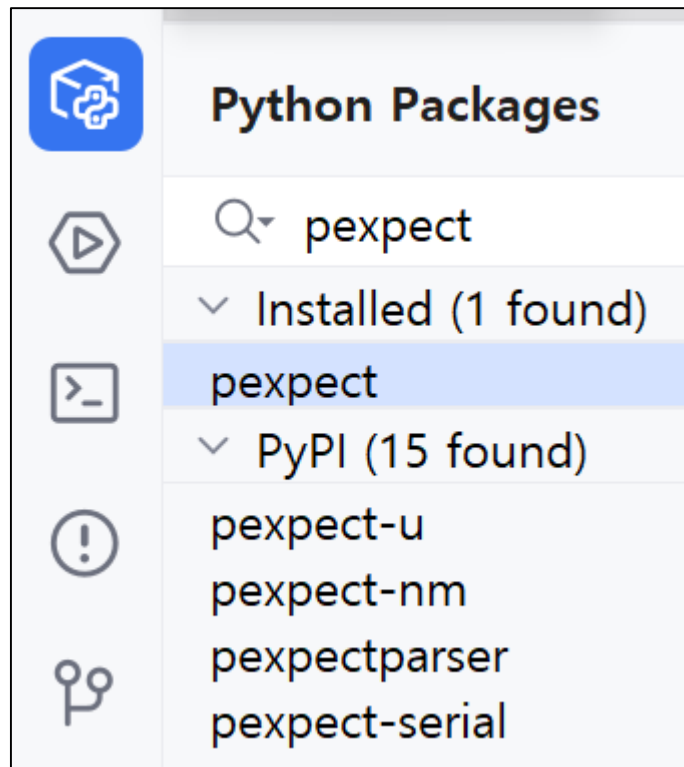
Chapter 06

Console 테스트 (E2E Test)

C++ 코드리뷰어를 위한 Build & Test 요약

pexpect 설치

Console E2E 테스트를 위해 pexpect를 설치한다.

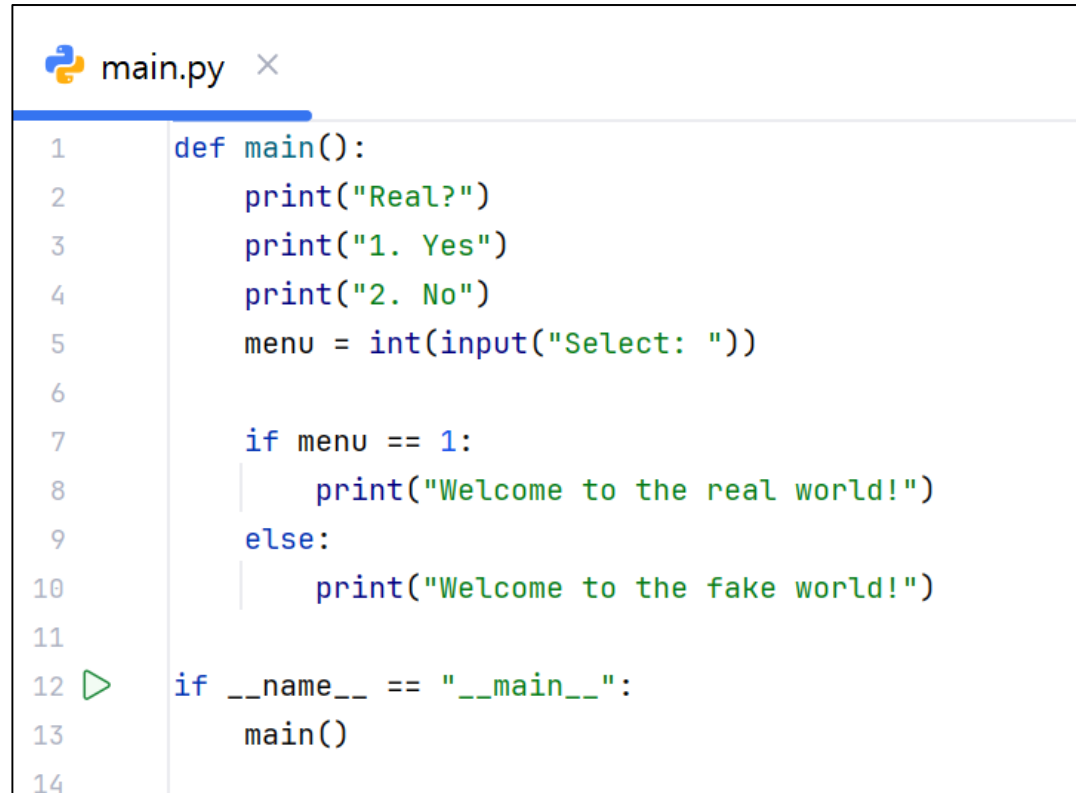
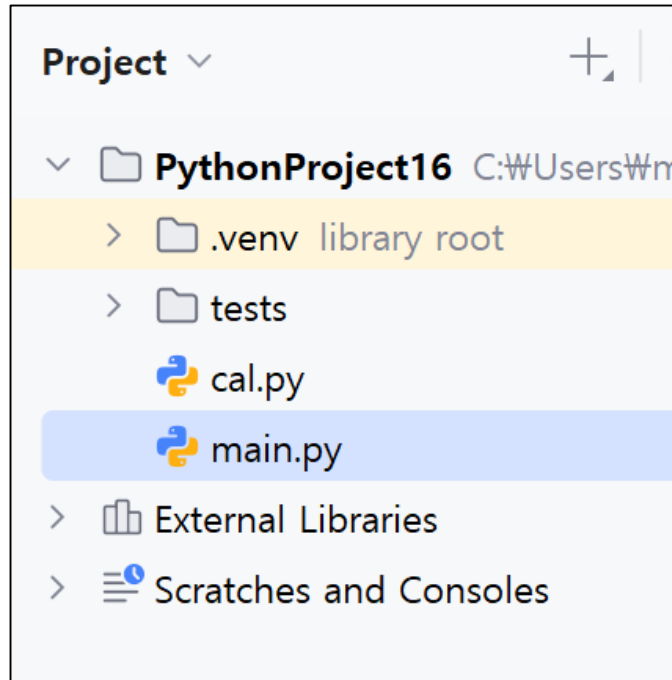


파일 추가 후 테스트 코드 삽입

E2E 테스트 대상이 될 기본 코드 추가하기

소스코드

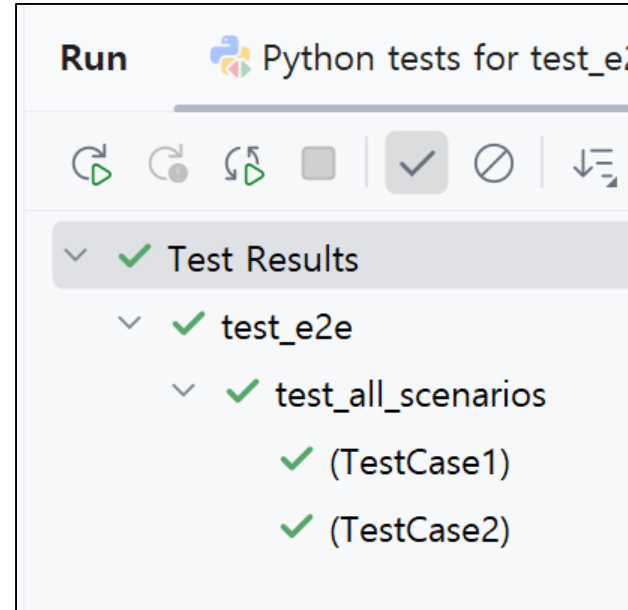
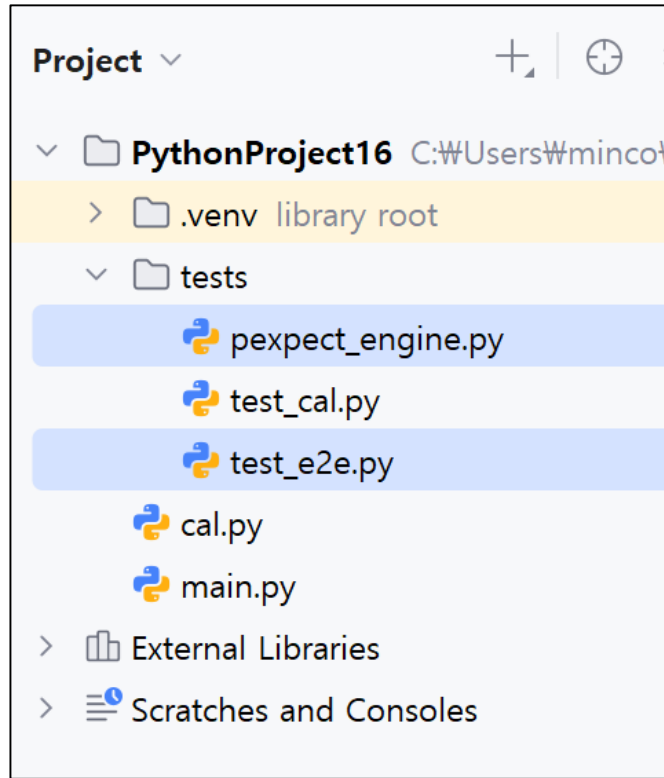
<https://gist.github.com/mincoding-ai/db03fac3f3ed0cde99d2a0272f5387fe>



E2E 테스트 코드 추가하기

test_e2e.py 파일 / pexpect_engine.py 생성 후, 소스코드 기입

<https://gist.github.com/mincoding-ai/db03fac3f3ed0cde99d2a0272f5387fe>



실행결과

감사합니다.